

L0.0-from-text-to-tokens

September 22, 2026

Master Degree in Computer Science and Data Science for Economics and Health

1 From text to tokens

1.1 A first look at how text becomes numbers

1.1.1 Prof. Alfio Ferrara

1.2 What do we want to do with text?

Think about the operational tasks we eventually want a machine to perform on text:

- **search** a collection of documents for the ones relevant to a query
- **classify** a document (spam/not spam, positive/negative review, cuisine of a recipe...)
- **cluster** documents that are “similar” to each other
- **generate** new text

All of these tasks share a common requirement: the machine needs a way to **compare** texts, to say that two texts are similar, different, or related. But a computer does not read; it only stores sequences of bytes. Before we can do anything *operational*, we need to turn text into something a machine can manipulate: numbers, vectors, geometry.

This notebook is about the first, most basic tension in that process.

1.3 The running example

We use a subset of the [Food.com Recipes and Reviews](#) dataset: about 10.000 recipes, each with a title, a cuisine, a type of dish and a free-text list of instructions.

```
import json

with open('/Users/Flint/Data/recipes/foodcom/subsets/recipes_cuisine_tipo_10k.json') as f:
    recipes = json.load(f)

print(f"Number of recipes: {len(recipes)}")
recipes[0]
```

Number of recipes: 9951

```
{'recipe_id': 285323,
 'title': "Victor's Cafe 52 Cuban Black Beans and Rice",
 'cuisine': 'Caribbean',
```

```

'type': None,
'ingredients': ['black beans',
'water',
'green pepper',
'onion',
'garlic cloves',
'green pepper',
'olive oil',
'salt',
'black pepper',
'oregano',
'bay leaf',
'sugar',
'vinegar',
'white wine',
'olive oil',
'olive oil',
'garlic cloves',
'water',
'salt',
'rice'],
'instructions': 'Wash beans. Soak them in water with 1 large green pepper, sliced, until tender, about 8 hours. Cook beans in same water for 45
minutes. In medium skillet, saute sliced onion and
remaining green pepper in 2/3 cup olive oil. Once soft, add this to the beans with salt, pepper, oregano, bay leaf, and sugar. Simmer for 1 hour.
Add vinegar and wine; simmer for another hour.
Meanwhile, in a flat pan, heat 3 T. oil. Fry garlic until it is light brown, then remove garlic from oil and discard. Add water and salt to the oil
in pan and bring to a boil. Add rice immediately.
Lower flame, cover, and simmer over low heat for about 30 minutes. When ready to serve beans, stir in 2 t. olive oil and serve the beans over white
rice. This recipe can be halved easily or the
leftovers can be frozen.'}

```

1.4 What the machine actually sees

For us, a recipe title or a set of instructions is meaningful. For the machine, it is just a string: a sequence of characters, with no notion of “word” or “meaning” attached to it.

```

sample = recipes[0]
print(sample['title'])
print(sample['instructions'][:200])
print()
print("...and this is what it actually is, character by character:")
print(list(sample['title']))

```

Victor's Cafe 52 Cuban Black Beans and Rice

Wash beans. Soak them in water with 1 large green pepper, sliced, until tender, about 8 hours. Cook beans in same water for 45 minutes. In medium
skillet, saute sliced onion and remaining green pepp

...and this is what it actually is, character by character:

```
['V', 'i', 'c', 't', 'o', 'r', '', 's', ' ', 'C', 'a', 'f', 'e', ' ', '5', '2', ' ', 'C', 'u', 'b', 'a', 'n', ' ', 'B', 'l', 'a', 'c', 'k', ' ', 'B',  
→ 'e', 'a', 'n', 's', ' ', 'a', 'n', 'd', ' ', 'R',  
'i', 'c', 'e']
```

So, how do we turn a bunch of characters into something we can compute with — something we can compare, measure a distance on, feed to a classifier? The natural answer is: **turn every text into a vector**, i.e. a point in some numeric space. Then “similar texts” becomes “vectors that are close together”, which we know how to compute (cosine similarity, euclidean distance, and so on).

The question becomes: **what do we count, to build that vector?** Let’s explore two opposite, extreme answers.

1.5 Extreme 1 — the whole document is the unit

The simplest possible feature we can extract from a text is the text itself: is this document *exactly* this string, yes or no? If we do this for every document in the corpus, we get one dimension per document: document i is the one-hot vector e_i , all zeros except a 1 in position i .

Let’s check what this means for two recipes that are, content-wise, extremely close to each other — two dessert recipes that only differ in a couple of ingredients and words in the instructions.

```
import numpy as np

desserts = [r for r in recipes if r['type'] == 'Dessert'][:2]
for r in desserts:
    print(r['title'])
    print(r['instructions'][:180])
    print()
```

Cashew Nut Cake(Aruba)

Cake: Cream margarine and sugar until smooth. Add egg whites and mix well. Add the rest of the cake ingredients and mix well. Grease a 9 round cake pan
→ and bake at 350°F for 25 mi

Diplomatic Pudding

In a food processor finely hop the bread and place in a large bowl. Mix the remaining ingredients. Stir well to dissolve the sugar. Place the mixture
→ in a caramel covered mold. Co

```
def one_hot_space(texts):
    """One dimension per distinct document: document-as-atom encoding."""
    n = len(texts)
    return np.eye(n)

texts = [r['instructions'] for r in desserts]
V = one_hot_space(texts)
print(f"Dimensionality of the space: {V.shape[1]} (one axis per document)")
print(f"Cosine similarity between the two (very similar!) dessert recipes: {V[0] @ V[1]:.2f}")
```

Dimensionality of the space: 2 (one axis per document)

Cosine similarity between the two (very similar!) dessert recipes: 0.00

No matter how similar two texts are, as long as they are not *character-for-character identical*, this representation places them on two orthogonal axes: maximum distance, zero similarity. And the number of dimensions grows with the size of the corpus — with 10.000 recipes we already have a 10.000-dimensional space, and it keeps growing as we add documents.

This is clearly useless for any of the tasks we listed above: it cannot generalize, and it does not scale. The problem is that the unit we chose to build features from — *the whole document* — is too large and too specific. So let’s go to the opposite extreme.

1.6 Extreme 2 — the character is the unit

Instead of the whole document, let’s use the smallest possible unit of text: the single character. We can represent every document as a vector of character frequencies (a “bag of characters”): the dimensionality of the space is now just the size of the alphabet used in the corpus, regardless of how many documents we have.

From now on we use the **ingredient list** of each recipe as the text to compare (rather than the instructions), because it is short and topic-loaded — a good candidate for telling recipes apart.

```
from collections import Counter

def ingredients_text(recipe):
    return " ".join(recipe['ingredients'])

def char_vector(text, vocabulary):
    counts = Counter(text.lower())
    total = sum(counts.values())
    return np.array([counts.get(c, 0) / total for c in vocabulary])

alphabet = sorted(set("".join(ingredients_text(r).lower() for r in recipes[:500])))
print(f"Dimensionality of the space: {len(alphabet)} (one axis per character)")
alphabet
```

Dimensionality of the space: 36 (one axis per character)

```
[' ',
 '%',
 '&',
 '"',
 '(',
 ')',
 ',',
 '-',
 '2',
 '9',
 'a',
 'b',
 'c',
 'd',
 'e',
 'f',
 'g',
 'h',
 'i',
 'j',
 'k',
```

```
'l',  
'm',  
'n',  
'o',  
'p',  
'q',  
'r',  
's',  
't',  
'u',  
'v',  
'w',  
'x',  
'y',  
'z']
```

Now let's compare three recipes: the two similar desserts from before, and one recipe that has nothing to do with them — a soup.

```
soup = [r for r in recipes if r['type'] == 'Soup/Stew'][0]  
print(soup['title'])  
print(soup['ingredients'])  
print()  
print(char_vector(soup['title'], alphabet))
```

Rice and Peas With Ham

```
['smoked ham', 'salt', 'garlic', 'ground cloves', 'tomatoes', 'green bell pepper', 'onion', 'long-grain rice', 'water']
```

```
[0.18181818 0.          0.          0.          0.          0.  
 0.          0.          0.          0.          0.13636364 0.  
 0.04545455 0.04545455 0.09090909 0.          0.          0.09090909  
 0.09090909 0.          0.          0.          0.04545455 0.04545455  
 0.          0.04545455 0.          0.04545455 0.04545455 0.04545455  
 0.          0.          0.04545455 0.          0.          0.          ]
```

```
from numpy.linalg import norm
```

```
def cosine(u, v):  
    return u @ v / (norm(u) * norm(v))
```

```
vecs = {  
    desserts[0]['title']: char_vector(ingredients_text(desserts[0]), alphabet),  
    desserts[1]['title']: char_vector(ingredients_text(desserts[1]), alphabet),  
    soup['title']: char_vector(ingredients_text(soup), alphabet),  
}
```

```
names = list(vecs.keys())  
print(f"similarity(dessert 1, dessert 2) = {cosine(vecs[names[0]], vecs[names[1]]):.4f}    <- similar recipes")
```

```
print(f"similarity(dessert 1, soup)      = {cosine(vecs[names[0]], vecs[names[2]]):.4f}  <- unrelated recipes")
print(f"similarity(dessert 2, soup)      = {cosine(vecs[names[1]], vecs[names[2]]):.4f}  <- unrelated recipes")
```

```
similarity(dessert 1, dessert 2) = 0.9468 <- similar recipes
similarity(dessert 1, soup)      = 0.9129 <- unrelated recipes
similarity(dessert 2, soup)      = 0.8687 <- unrelated recipes
```

The three similarities are all high and close together (well above 0.85), with no clear gap between the pair we know is similar and the pairs we know are not. This is not a coincidence: the relative frequency of letters in a language is remarkably stable across texts, no matter what they are about. A cake’s ingredient list and a soup’s ingredient list use “e”, “a”, “t”, “space” with roughly the same frequencies, because they are both written in English. By shrinking the unit down to single characters, we gained a tiny, corpus-independent space — but we lost almost all the ability to tell texts apart. Everything **collapses**.

We have two extremes:

unit	dimensionality	discriminative power
whole document	huge, grows with corpus size	perfect, but generalizes to nothing (everything is orthogonal)
single character	tiny, fixed	none, everything collapses together

Neither extreme gives us a space where similar texts are close and different texts are far apart, which is exactly the property we need for search, classification, clustering... We need a unit **in between**.

1.7 In between — the word

Let’s try the same experiment on the same ingredient lists, but using words as the unit instead of characters or whole documents: a “bag of words” vector, counting how many times each word occurs in the text.

```
def word_vector(text, vocabulary):
    counts = Counter(text.lower().split())
    total = sum(counts.values())
    return np.array([counts.get(w, 0) / total for w in vocabulary])

vocabulary = sorted(set(" ".join(ingredients_text(r).lower() for r in recipes[:1000]).split()))
print(f"Dimensionality of the space: {len(vocabulary)} (one axis per distinct word)")
```

Dimensionality of the space: 779 (one axis per distinct word)

```
vecs = {
    desserts[0]['title']: word_vector(ingredients_text(desserts[0]), vocabulary),
    desserts[1]['title']: word_vector(ingredients_text(desserts[1]), vocabulary),
    soup['title']: word_vector(ingredients_text(soup), vocabulary),
}

print(f"similarity(dessert 1, dessert 2) = {cosine(vecs[names[0]], vecs[names[1]]):.4f}  <- similar recipes")
print(f"similarity(dessert 1, soup)      = {cosine(vecs[names[0]], vecs[names[2]]):.4f}  <- unrelated recipes")
print(f"similarity(dessert 2, soup)      = {cosine(vecs[names[1]], vecs[names[2]]):.4f}  <- unrelated recipes")
```

```
similarity(dessert 1, dessert 2) = 0.3944 <- similar recipes
similarity(dessert 1, soup)      = 0.1166 <- unrelated recipes
similarity(dessert 2, soup)      = 0.2070 <- unrelated recipes
```

Already with this very crude word-counting scheme, the two similar desserts turn out closer to each other than either of them is to the soup. The dimensionality is still large (it grows with the vocabulary of the corpus, not as fast as with whole documents, but faster than with characters), yet the space now has some *structure*: proximity means something.

This is not an accident: the word is a unit that is small enough to recur across many documents (so a text can share dimensions with other texts, unlike whole-document encoding), but large enough to carry a piece of meaning (unlike single characters). It sits at the right point in the trade-off.

1.8 From word to token

We used “word” so far, defined in the crudest possible way — splitting on whitespace. But this raises immediate questions:

- what do we do with punctuation, capitalization, “don’t” vs “do not”, plurals, verb conjugations?
- what about languages without whitespace between words, or with very rich morphology, where a “word” can be a whole sentence glued together?
- what about a brand-new word the model has never seen — do we throw it away?

Because of all this, in NLP we rarely commit to “word” as *the* unit. Instead we use the more general term **token**: whatever unit we decide to split text into, following some rule or some learned procedure. A token might be a word, a piece of a word, a single character, or even a whole common phrase — the point is that it is *the* unit our numeric representation is built on.

We have only shown *why* we need an intermediate unit between “whole document” and “single character”. We have **not** yet discussed *how* to actually split a text into tokens — that is the subject of the next lesson.